PLAN DE TESTS

NiceSpot – Right to the Nice Spot

Application	NiceSpot
Slogan	Right to the Nice Spot
Version du document	1.0
Date de rédaction	22/03/2026
Rédigé par	Ibrahim Ari Malla
Relu par	
Statut	En cours de rédaction

SOMMAIRE

1.	Points d'évaluation – Criticité, enjeux client & business
2.	Stratégie & Types de tests – Matrice fonctionnalité × test
3.	Critères de réussite – Seuils, liste des tests
4.	Maintenance & Bonnes pratiques – Règles de conduite

1. Points d'évaluation

Objectif : classer les fonctionnalités par criticité, enjeu client et enjeu business pour justifier ce qui doit être testé en priorité.

A. Fonctionnalités existantes (V1 – en production)

#	Fonctionnalité	Description	Criticité	Enjeu client	Enjeu business	Priorité
1	**Authentification**	Connexion / inscription + contrôle d'accès par token	Critique	Élevé	Élevé	● **Haute**
2	**Ajout de lieux**	Formulaire + géocodage + enregistrement BDD	Critique	Élevé	Élevé	● **Haute**
3	**Affichage carte**	Carte interactive avec marqueurs géolocalisés	Critique	Élevé	Élevé	● **Haute**
4	**Liste & filtres**	Affichage liste + filtres (mes lieux / autres)	Élevé	Élevé	Moyen	● **Moyenne**
5	**Priorité VIP**	Lieux VIP affichés en tête de liste	Élevé	Moyen	Élevé	● **Haute**
6	**Recherche géo**	Tri de la liste par proximité géographique	Moyen	Élevé	Moyen	● **Moyenne**

B. Fonctionnalités à venir (hors scope V1)

#	Fonctionnalité	Description	Criticité	Enjeu client	Enjeu business	Priorité
1	**Notation (0–10)**	Score utilisateur sur chaque lieu	Élevé	Élevé	Élevé	● **Moyenne**
2	**Commentaires**	Lecture + ajout de commentaires	Élevé	Élevé	Moyen	● **Moyenne**
3	**Photos**	Upload et affichage de photos des lieux	Moyen	Élevé	Moyen	● **Moyenne**
4	**Navigation Google Maps**	Ouverture Maps avec coordonnées du lieu	Moyen	Élevé	Faible	◍ **Basse**
5	**Paiement / VIP**	Débit en échange de services premium	Élevé	Moyen	Élevé	● **Moyenne**
6	**Géolocalisation mobile**	Repérage + notification lieux proches	Moyen	Élevé	Moyen	◍ **Basse**
7	**Personnalisation**	Modifier profil / adresse, décorations	Faible	Moyen	Faible	◍ **Basse**
8	**Administration**	Suspendre users, dashboard statistiques	Critique	Faible	Élevé	● **Haute**

● **Haute**	Fonctionnalité bloquante – doit être testée avant tout déploiement
● **Moyenne**	Fonctionnalité importante – testée à chaque release significative
◍ **Basse**	Fonctionnalité secondaire – testée selon disponibilité / roadmap

2. Stratégie & Types de tests

Objectif : définir quels types de tests sont adaptés à chaque fonctionnalité et justifier les choix pour optimiser la stratégie.

A. Types de tests retenus

Code	Type	Périmètre	Automatisé ?	Outil suggéré	Coût	Quand l'utiliser
UT	**Tests unitaires**	Fonction / méthode isolée	☑ Oui	Jest	Faible	Chaque nouvelle fonction ou service
IT	**Tests d'intégration**	Interaction modules / API + BDD	☑ Oui	Supertest	Moyen	Routes API, accès BDD, services composés
E2E	**Tests End-to-End**	Parcours utilisateur complet	☑ Oui	Playwright	Élevé	Parcours critiques : auth, ajout lieu, paiement
MAN	**Tests manuels**	UI, UX, multi-devices	✕ Non	Cahier de recette Excel	Moyen	Avant chaque release, via les parties prenantes

B. Matrice fonctionnalité × type de test

#	Fonctionnalité	UT – Unitaires	IT – Intégration	E2E	Manuel	Justification
1	**Authentification**	☑ Token, hash mdp	☑ /login, /register	☑ Parcours complet	☑ Multi-navs	Critique – couverture maximale
2	**Ajout de lieux**	☑ Validation form	☑ Enregistrement BDD	☑ Form → confirmation	☑ UX formulaire	Cœur du produit – E2E indispensable
3	**Affichage carte**	✕ Difficile en UT	✕	☑ Affichage marqueurs	☑ Rendu visuel	Rendu visuel = E2E + manuel
4	**Liste & filtres**	☑ Logique tri/filtre	☑ Query BDD filtrée	☑ Filtres interactifs	☑ Vérif VIP	Filtres = IT + E2E
5	**Recherche géo**	☑ Calcul distance	☑ Query triée	☑ Résultats triés	☑	Logique métier = UT + IT
6	**Priorité VIP**	☑ Logique rôle	☑ BDD + API	✕	☑ Vérif visuelle	Enjeu business fort
7	**Paiement (futur)**	☑ Calculs montants	☑ API paiement test	☑ Parcours achat	☑	Critique – couverture maximale
8	**Administration (futur)**	☑ Droits & rôles	☑ Actions admin	☑ Dashboard	☑	Sécurité = priorité absolue

3. Critères de réussite

Objectif : définir à quel moment les tests sont considérés comme validés et l'application prête à être déployée.

A. Seuils de validation globaux

#	Critère	Seuil requis	Seuil bloquant (no-go)	Outil de mesure	Statut
1	**Taux de réussite – Tests unitaires**	100 %	< 100 % = déploiement bloqué	Jest	À définir
2	**Taux de réussite – Tests d'intégration**	100 %	< 100 % = déploiement bloqué	Supertest	À définir
3	**Taux de réussite – Tests E2E**	100 % parcours critiques	Tout échec auth/paiement bloque	Playwright	À définir
4	**Code coverage global**	≥ 80 %	< 60 % = bloquant	Jest --coverage	À définir
5	**Coverage fonctions critiques**	≥ 95 % (auth, paiement)	< 80 % = bloquant	Jest --coverage	À définir
6	**Temps d'exécution pipeline CI**	< 5 min	> 10 min = optimisation requise	GitHub Actions	À définir
7	**Cahier de recette manuel**	0 KO parcours critiques	1 KO = no-go déploiement	Excel recette	À définir

B. Liste des tests à implémenter

ID	Fonctionnalité	Type	Nom / description du test	Critère de validation	Implémenté ?
UT-01	Authentification	Unitaire	getUserFromRequest()	Retourne les infos de l'utilisateur si token valide	☑ Oui
UT-02	Authentification	Unitaire	hashPassword()	Hash !== mot de passe brut	✕ À faire
UT-03	Ajout de lieux	Unitaire	validateForm()	Erreur si champ obligatoire vide	✕ À faire
UT-04	Recherche géo	Unitaire	getDistance()	Retourne la différence entre deux points	☑ Oui
IT-01	Authentification	Intégration	POST /api/users/token – identifiants valides	200 OK + token JWT retourné	☑ Oui
IT-02	Authentification	Intégration	Inscription avec un email déjà existant	403 Forbidden	☑ Oui
IT-03	Ajout de lieux	Intégration	POST /places – données valides	201 Created + lieu en BDD	✕ À faire
IT-04	Liste lieux	Intégration	GET /places?filter=mine	Uniquement les lieux de l'user	✕ À faire
E2E-01	Authentification	E2E	Parcours connexion complet	Redirection + token stocké	☑ Oui
E2E-02	Ajout de lieux	E2E	Formulaire → soumission → confirmation	Lieu visible sur carte	✕ À faire
E2E-03	Paiement	E2E	Parcours achat abonnement VIP	Compte passé en VIP	✕ À faire

4. Maintenance & Bonnes pratiques

Ces règles doivent être connues et appliquées par toute l'équipe. Elles évoluent avec le projet.

A. Chronologie type d'une release

Étape	Action	Détail	Responsable
1	**Développement**	Implémentation de la fonctionnalité avec ses tests unitaires et d'intégration	Dev
2	**Revue de code**	Pair review + vérification que les nouveaux tests couvrent bien la fonctionnalité	Tech Lead
3	**CI automatique**	Pipeline déclenché : UT + IT + E2E + coverage. Bloque si seuils non atteints	CI/CD (auto)
4	**MAJ cahier de recette**	Mise à jour du cahier et diffusion aux parties prenantes avec deadline d'exécution	QA
5	**Phase de recettage**	Les parties prenantes exécutent les tests manuels et remontent les retours	Parties prenantes
6	**Analyse des retours**	Traitement des KO, corrections et nouvelle exécution si nécessaire	QA + Dev
7	**Go / No-Go**	Si tous les critères sont atteints → déploiement autorisé	Tech Lead / PO

B. Règles de conduite & bonnes pratiques

#	Règle	Description détaillée	Niveau
1	**Tests exécutés automatiquement**	L'exécution des tests NE DOIT PAS être optionnelle. Le pipeline CI bloque toute PR non conforme aux seuils définis.	● **Obligatoire**
2	**Zéro contournement de test**	Un test qui échoue = correction ou mise à jour du test. Jamais de skip sans justification écrite validée par le Tech Lead.	● **Obligatoire**
3	**Tout nouveau code = nouveau test**	Toute fonctionnalité ajoutée doit être accompagnée de ses tests (UT minimum). Aucune PR sans couverture.	● **Obligatoire**
4	**Tests sur données isolées**	Utiliser des fixtures / seeds de test. Jamais de tests sur données de production.	● **Obligatoire**
5	**Mise à jour du plan de tests**	Le plan de tests est mis à jour à chaque nouvelle fonctionnalité ou changement de périmètre.	● **Fortement recommandé**
6	**Mise à jour du cahier de recette**	Le cahier est revu et mis à jour avant chaque diffusion aux parties prenantes.	● **Fortement recommandé**
7	**Conservation des rapports**	Les résultats de tests sont archivés par version pour détecter des régressions dans le temps.	● **Fortement recommandé**
8	**Diffusion de la culture test**	Les bonnes pratiques sont partagées en équipe lors de l'onboarding, des revues de code et des rétrospectives.	◍ **Recommandé**